DC-URS v1 — implementation plan

Contract: `dc-urs-spec.md` **Date:** 2026-07-27 **Estimate:** ~3 days across five phases, each landing green before the next.

Every phase ends with `npm run verify` passing and a commit. Phase 0 is independent of all data acquisition and can start immediately.

Audit findings — read before planning anything

The calibration plan was audited against the code before execution and four assumptions turned out wrong. Same exercise here; two findings change the plan materially.

The replacement surface is much smaller than expected

`greenScore` has **exactly one call site** — `heat-map-app.ts:350` — and only three files mention it at all: the model, the app, and the Astro shell. No tests, no compare page, no scenario URL encoding. The swap is genuinely localised.

The UI surface is also close to a 1:1 match. The existing `.subs` strip shows **three raw sub-scores** because a composite index is only auditable if you can see which component produced the number. DC-URS has **three pillars**. The strip becomes ACI / EVI / THI with almost no layout work.

DC-URS has no economics, and that is a real product regression

The Green Score's third component is **budget efficiency** — °C per crore. That is why trees score 84 and facades score 4: the index rewards cost-effectiveness, and that ordering matches Indian Heat Action Plan practice.

DC-URS contains no cost term at all. `ACI = UGS + CRI + TRA`; nothing in the index knows what anything costs. So on replacement:

- a ward that achieves its greenery cheaply and one that spends ₹200 crore to get there **score identically**
- the "which intervention is best value" answer — arguably the most decision-useful thing the tool produces — disappears from the score

This is not a defect in DC-URS. It is a *state* index, not an *efficiency* index, and it is correct for that job. But the capability is lost unless we deliberately keep it. **Flagged to the CEO as a decision** (§Open questions); the plan below keeps cost computed and displayed so the option stays open, and adds nothing to the index without his sign-off.

Smaller findings

- `WARDS` is **inline in** `heat-map-app.ts`, not in `src/data/`. Spec §1 requires it to be data. Extraction is Phase 0 work, not a refactor to do later.
- `state.greenG` (`computeGreenG`) feeds the Green Score's greening ratio. DC-URS does not use it, but it is closely related to `FVC` and should **not** be deleted until Phase 4 confirms nothing else reads it.
- `scoreTxt` currently renders cooling, greening and cost together. It needs rewriting regardless of the cost decision.

File plan

```
src/data/
  wards.ts                        NEW  P0 · ward list extracted from heat-map-app.ts

src/scripts/climate-engine/
  dc-urs.ts                       NEW  P0 · the engine, pure and self-checked
  dc-urs-inputs.ts                NEW  P0 · typed indicator record + per-field provenance
  heat-map-app.ts                 EDIT P4 · swap the call site, wire the pillars
  heat-map-model.ts               EDIT P4 · retire greenScore/greenScoreParts

src/components/ClimateEngine/
  HeatMapStage.astro              EDIT P4 · sub-scores → pillars, structural floor, tier
label

scripts/
  fetch-worldpop.py               NEW  P2 · population density per footprint
  fetch-socio.py                  NEW  P2 · HVI_socio — NFHS-5 levels x Census 2011
pattern
  fetch-sentinel-composites.py    NEW  P1 · seasonal FVC, albedo, NDVI series (keyless
STAC)
  compute-far.py                  NEW  P1 · FAR from Google 2.5D + MS footprints
  compute-tra.py                  NEW  P1 · distance-to-refuge from OSM
  build-dcurs-inputs.py           NEW  P1 · assemble → data/dc-urs/inputs.json
  measure-dcurs-uncertainty.py    NEW  P3 · Monte Carlo band
  validate-dcurs.mjs              NEW  P3 · the four spec §7 checks

data/dc-urs/
  inputs.json                     NEW  P1 · one record per ward, every field with
provenance
  golden-cases.json               NEW  P0 · frozen regression fixture
```

`dc-urs.ts` is separate from `heat-map-model.ts` because it is a different kind of thing: the model computes physics, the index composes indicators. Mixing them is what made the Green Score hard to audit.

Phase 0 — The engine (½ day) — DONE 2026-07-27, commit 73669f7

No data acquisition. Everything here runs on hand-fed inputs.

0a · `src/data/wards.ts`

- [ ] Move the `WARDS` object out of `heat-map-app.ts` verbatim, add the fields DC-URS needs.

```
export interface Ward {
  readonly id: string;
  readonly name: string;
  readonly zone: string;
  readonly body: string;          // KMC Ward 68 | Baruipur Municipality | …
  readonly lat: number;
  readonly lon: number;
  readonly footprintM: number;    // 1400 — matches the thermal simulation domain
}

export const WARDS: readonly Ward[] = [ /* … */ ];
```

- [ ] `heat-map-app.ts` imports from here. **Adding a ward must never require touching a script.**

0b · `src/scripts/climate-engine/dc-urs-inputs.ts`

- [ ] The eleven indicator values with provenance per field, reusing the `Sourced<T>` idea already established in the project.

```
export type Provenance = 'measured' | 'modelled' | 'estimated' | 'placeholder';
export interface Sourced<T> { value: T; source: Provenance; vintage?: string; cite?:
string }

export interface DcUrsInputs {
  lstDayC: Sourced<number>;   lstNightC: Sourced<number>;  ruralBaseC: Sourced<number>;
  fvc: Sourced<number>;       canopyFrac: Sourced<number>;
  ndviMean: Sourced<number>;  ndviStd: Sourced<number>;
  albedo: Sourced<number>;    distCoolM: Sourced<number>;
  popDensity: Sourced<number>; far: Sourced<number>; socioVuln: Sourced<number>;
}
```

Provenance is per field because it lands field by field, and the UI must mark what is 2011 census versus what is this morning's satellite pass.

0c · `src/scripts/climate-engine/dc-urs.ts`

- [ ] Anchors as named constants, each carrying its provenance in a comment (spec §4).
- [ ] The corrected engine, exactly as spec §2:

```
/** Vegetation stability. Guarded: water's negative mean NDVI made the unguarded
 *  form return 1.00 — perfect vegetation stability for a river. */
export function vsi(ndviMean: number, ndviStd: number): number {
  if (ndviMean <= VSI_NDVI_FLOOR) return 0;          // 0.10
  return 1 - clamp(ndviStd / ndviMean, 0, 1);
}

export function pillars(i: DcUrsInputs): { aci: number; evi: number; thi: number; ugs:
number } {
  const thi = 0.40 * clamp((i.lstDayC.value - 25) / 25, 0, 1)      // /25, not /20 —
delta 7
            + 0.40 * clamp((i.lstNightC.value - 20) / 15, 0, 1)
            + 0.20 * clamp(Math.max(0, i.lstDayC.value - i.ruralBaseC.value) / 10, 0, 1);
  const evi = 0.45 * clamp(i.popDensity.value / 25000, 0, 1)
            + 0.30 * clamp(i.far.value / 5, 0, 1)
            + 0.25 * clamp(i.socioVuln.value / 10, 0, 1);
  const ugs = 0.50 * clamp(i.fvc.value, 0, 1)
            + 0.30 * clamp(i.canopyFrac.value, 0, 1)
            + 0.20 * vsi(i.ndviMean.value, i.ndviStd.value);
  const aci = 0.50 * ugs + 0.30 * clamp(i.albedo.value / 0.60, 0, 1)
            + 0.20 * Math.exp(-0.002 * i.distCoolM.value);
  return { aci, evi, thi, ugs };
}

/** Geometric, not additive. IPCC AR6 risk is conjunctive: a near-zero pillar must
 *  drag the score down rather than be bought off by a strong one. */
export function dcUrs(i: DcUrsInputs): number {
  const { aci, evi, thi } = pillars(i);
  const g = (x: number) => clamp(x, 0.001, 1);       // never evaluate 0^0.4
  return 100 * g(aci) ** 0.40 * g(1 - evi) ** 0.35 * g(1 - thi) ** 0.25;
}
```

- [ ] `tierFor(score)` returning the spec §6 band, label and colour.
- [ ] `structuralFloor(i)` — a **named export**, not an incidental number:

```
/** Points that no intervention can recover, because EVI responds to nothing the
 *  user can do. This is the tool's sharpest output, not a limitation: it separates
 *  what greening can fix from what only housing and health policy can. */
export function structuralFloor(i: DcUrsInputs): { ceiling: number; withheld: number } {
  const perfect: DcUrsInputs = { ...i,
    fvc: { ...i.fvc, value: 1 }, canopyFrac: { ...i.canopyFrac, value: 1 },
    ndviMean: { ...i.ndviMean, value: 0.85 }, ndviStd: { ...i.ndviStd, value: 0.03 },
    albedo: { ...i.albedo, value: 0.60 }, distCoolM: { ...i.distCoolM, value: 0 },
    lstDayC: { ...i.lstDayC, value: i.ruralBaseC.value },
    lstNightC: { ...i.lstNightC, value: i.lstNightC.value - 5 },
  };
  const ceiling = dcUrs(perfect);
  return { ceiling, withheld: 100 - ceiling };
}
```

0d · Self-check and golden cases

- [ ] `assertDcUrsLogic()` — runnable with `node --experimental-strip-types`:

```
export function assertDcUrsLogic(): void {
  const a = (ok: boolean, m: string) => { if (!ok) throw new Error(`dc-urs: ${m}`); };
  a(vsi(-0.30, 0.05) === 0, 'water must score 0 stability, not 1');
  a(vsi(0.05, 0.06) === 0, 'bare ground must score 0 stability');
  a(vsi(0.75, 0.04) > 0.9, 'mature canopy must score high stability');
  // bounded, and monotone in each pillar
  for (const w of GOLDEN) {
    const s = dcUrs(w.inputs);
    a(s >= 0 && s <= 100, `${w.id}: score out of range`);
    a(Math.abs(s - w.expected) < 0.05, `${w.id}: ${s.toFixed(2)} vs frozen
${w.expected}`);
  }
  // a lethal pillar cannot be bought off — the whole point of geometric aggregation
  a(dcUrs(HOT_BUT_GREEN) < dcUrs(COOL_BUT_BARE), 'hot ward must not outscore cool ward');
  a(structuralFloor(BALLYGUNGE).ceiling < 100, 'structural floor must withhold
something');
}
```

- [ ] Freeze `data/dc-urs/golden-cases.json` from the dry run already performed: **Ballygunge 21.13**, **Baruipur 64.16**.

Barrackpore is in scope everywhere else — it is one of the three study wards and Phases 1, 2 and 4 all cover it. It is held out of *this fixture only*, and only until Phase 1. The reason: Ballygunge and Baruipur have real inputs, worked out in the CEO's own specification. Barrackpore has none, so its dry-run 40.14 was computed from **invented** inputs (`fvc 0.26`, `canopy 0.20`, `socio 6.0`, …) chosen as plausible for an industrial corridor, purely to check that geometric aggregation kept three wards in three distinct tiers. A golden case is a frozen assertion that a number is *correct*; freezing estimates would test every future change against a fiction. **Barrackpore joins the fixture the moment Phase 1 supplies measured inputs.**

Verify: `assertDcUrsLogic()` passes; `npm run check` 0 errors. Nothing user-visible changes yet.

Outcome. Golden cases reproduced exactly — Ballygunge **21.13**, Baruipur **64.16**. `npm run verify` green (0 errors, 26/26 unit tests, publication contract 77/77).

One assertion written for this phase was **wrong**, and recording why matters: it claimed a ward at 47 °C must never outscore one at 31 °C, and it failed (43.2 vs 39.2). The engine was right. Geometric aggregation removes FULL compensation, not ALL compensation — it is still a weighted product, and `w_aci` (0.40) exceeds `w_thi` (0.25), so a 4.7x adaptive-capacity advantage can outweigh a large hazard disadvantage. That is the CEO's weighting choice. "Fixing" the engine to satisfy the test would have silently overridden his weights. The assertions now test what geometric aggregation actually guarantees, and the reasoning is written into the module.

Structural floor for Ballygunge under the geometric form is **harsher** than the additive dry run suggested: ceiling **54.8** (not 61.4), **45.2 points withheld**. A physically perfect retrofit cannot lift it out of "Vulnerable".

Phase 1 — Observed pillars (1 day)

Real satellite and vector data for everything except demographics.

- [ ] `fetch-sentinel-composites.py` — Sentinel-2 L2A via **keyless public STAC**. No Earth Engine: all three of Microsoft Planetary Computer, Copernicus Data Space and AWS Earth Search were verified live (HTTP 200, 2026-07-27). EE was only the path we held credentials for from the AlphaEarth work and is not part of the CEO's design; dropping it removes a credential to maintain and a readiness check to run. **Seasonal composites, never single scenes** (spec §3): Kolkata NDVI swings hard between monsoon and dry season. Emit per ward: `fvc`, `albedo`, and a ≥5-year annual NDVI series for `ndviMean` / `ndviStd`. Albedo uses the source document's coefficients — note in the header that this is a Liang-type narrowband-to-broadband conversion so the lineage is traceable.
- [ ] `compute-far.py` — floor-area ratio from Google Open Buildings 2.5D heights over Microsoft footprints, both already held. `FAR = Σ(footprint area × floors) / footprint area`, floors estimated at 3.2 m storey height, assumption stated in the output.
- [ ] `compute-tra.py` — nearest thermal refuge from OSM (`leisure=park`, `natural=water`, `amenity=community_centre`), distance transform over the ward footprint, mean distance.

- [ ] `canopyFrac` — ESA WorldCover class 10 fraction, reusing `suhii.align()` and the 70 m target grid so it lands on the same pixels as the thermal work.
- [ ] `build-dcurs-inputs.py` — assemble into `data/dc-urs/inputs.json`, one record per ward, **every field carrying its provenance and vintage**.

Verify: every observed field populated for all three wards; no field left `placeholder`; run the engine on real inputs and record the scores. Expect movement from the dry run — the dry run used the CEO's illustrative numbers.

Watch for: `canopyFrac` from a top-of-canopy product suppresses built fraction in dense tropical cities (street trees occlude roofs). Record the caveat; do not correct for it silently.

Phase 2 — Demographics (½ day)

- [ ] `fetch-worldpop.py` — WorldPop constrained UN-adjusted, clipped to each 1400 m footprint → `popDensity` in persons/km². CC BY 4.0, attribution recorded. **Latest confirmed release is 2020**, not 2021 — `ind_ppp_2020_UNadj_constrained.tif` and the 1 km density product both resolve (HTTP 200); check for a newer year at build time rather than assuming one exists.
- [ ] `fetch-socio.py` — `HVI_socio` from **two** sources, composed the Rathi (2021) / Azhar (2017) way (share >65, share <5, low-income proxy, informal-settlement fraction):
  - **NFHS-5 (2019–21)** for present-day levels — distributed via DHS Program (`dhsprogram.com`, HTTP 200; microdata needs free registration). Resolves to **district**.
  - **Census 2011** for the ward-level spatial pattern — the only enumeration at the geography DC-URS needs. Combine as district level × ward-relative pattern, then **areal-interpolate** onto the 1400 m footprint, because the three wards sit under three different local bodies and no single body's figures cover them (spec §1).
- [ ] Both vintages surfaced in `inputs.json` and rendered in the UI. **Census 2021 does not exist** — deferred to Census 2027, reference date 1 March 2027. NFHS-5 is the closest 2021-vintage source in existence, at coarser geography.

Verify: EVI complete for all three wards; population sanity-checked against published Kolkata ward densities; the WorldPop-vs-Census gap recorded rather than reconciled; the district-to-ward downscaling assumption stated explicitly in `inputs.json`, not buried in a script.

Phase 3 — Validation (½ day)

`validate-dcurs.mjs`, the four spec §7 checks. Rank correlation is **not** among them — it is impossible at n=3 and that is stated, not worked around.

- [ ] **Golden-case regression** — the frozen fixture must reproduce exactly.

- [ ] **Face validity** — ranking must match known Kolkata geography. A dense hot core outscoring a green fringe is a failure whatever the maths says.
- [ ] **Sensitivity analysis** — vary each of the eleven inputs across its plausible range, record the DC-URS swing, print a ranked table. **If one indicator dominates, the other ten are decoration and the weighting is wrong.** Report; do not silently retune.
- [ ] `measure-dcurs-uncertainty.py` — Monte Carlo the thermal model's measured ±3.5 °C night / ±5.0 °C day through the geometric form. The additive-era estimate of ~5 points **does not carry over** and must be re-measured (spec §6).

Verify: all four run and report. No numeric bar is set for face validity or sensitivity — inventing one at n=3 would repeat the ±2 K mistake from the thermal calibration, where a bar was set before anyone knew what was achievable.

Phase 4 — Wire it in, retire the Green Score (½ day)

- [ ] Replace the single call site at `heat-map-app.ts:350`. Baseline pillars come from `inputs.json`; scenario pillars apply modelled deltas to `fvc`, `canopyFrac`, `albedo`, `distCoolM` and the LST terms. `vsi` **is frozen at baseline** — newly planted cover scores ≈0.27 against mature canopy's 0.95, so a live VSI would make planting trees *lower* the score.
- [ ] `.subs` strip: greening/cooling/efficiency → **ACI / EVI / THI**, raw, same auditability rationale as before.
- [ ] Tier label and colour from `tierFor()`, including "Critical Hotspot" — CEO-approved wording.
- [ ] **Structural floor rendered**, not hidden: achievable headroom shown separately from withheld points, with the sentence that makes it useful — *"28 of this ward's missing points are demographic exposure; trees cannot fix them."*
- [ ] Uncertainty band beside the score, matching the treatment already shipped for temperature. Baseline uses observed temperature; scenario carries the model band. **The asymmetry is visible.**
- [ ] Delete `greenScore` / `greenScoreParts` from `heat-map-model.ts`. Keep `computeCost` and `computeGreenG` until the cost decision is made.
- [ ] Update `docs/green-score-methodology.md` — the index it documents no longer ships.

Verify: `npm run verify` green; e2e passes; sliders move DC-URS; no console errors; the score never leaves 0–100.

Open questions for the CEO

1. **Does the tool keep a cost-effectiveness signal?** DC-URS has no economics, so trees and green facades that achieve the same greenery score identically regardless of costing ₹66 lakh or ₹190 crore. Options: (a) accept the loss, cost shown as context only; (b) display ₹/point alongside the score

without touching the index; (c) add a fourth pillar — a change to his engine. **Plan assumes (b)** as the reversible default.

2. **What replaces `docs/green-score-methodology.md`?** The consultant reviewed it eight days ago. A short "what changed and why" note is probably owed.

What could go wrong

	Symptom	Response
Golden cases drift	Frozen values don't reproduce	A real regression. Do not re-freeze to make it pass
Sensitivity dominated by one input	One indicator moves the score, ten don't	Weighting problem. Report with the table; CEO decides
Seasonal NDVI ambiguity	FVC differs wildly by composite window	Fix the window in the spec and record it; do not average monsoon with dry
Census interpolation implausible	A ward's density is obviously wrong	Say so. Do not smooth it into looking reasonable
NFHS-5 district masks real ward variation	Three wards inherit near-identical socio values	Expected — NFHS-5 is district-resolution. Census 2011 supplies the within-district pattern; if that still flattens them, report it rather than inventing spread
Wards land in one tier	Real inputs compress the range	Report before adjusting anchors — the anchors are the CEO's and researched
Score jumps vs Green Score	Users see a different number	Expected and pre-agreed. Document the delta per ward
Uncertainty band is large	±10 points or worse	Publish it. That is the honest outcome and the precedent is set

---

Sequencing rule

One change at a time, measured after each. The failure this project has already seen twice — a constant tuned until the picture looked right — is prevented by the golden-case fixture and the sensitivity table, not by good intentions. Every phase ends with a number that either matches expectation or does not.